

Bachelor's thesis

Hardware Aware Task Modeling of Neural Network Inference for FreeRTOS

Kevin Alexander Butzke

Reviewers:

Prof. Dr. Jian-Jia Chen

Dr.-Ing. Christian Hakert

2024



Technische Universität Dortmund

Fakultät Informatik

Lehrstuhl 12 Design Automation for Embedded Systems Group

Hardware Aware Task Modeling of Neural Network Inference for FreeRTOS

Kevin Butzke ✉

TU Dortmund, Germany

Abstract

Efficient neural network inference on embedded systems is crucial due to the growing number of devices leveraging neural networks for tasks such as image classification, demanding optimized task management strategies. This thesis investigates optimizing FreeRTOS-based task execution for neural network layers to improve execution time and to highlight the impact of employing FreeRTOS tasks within a neural network context. The optimization approach involves modeling neural networks layers as FreeRTOS tasks, using task notifications and semaphores for synchronization, and multi-core execution of layers or batches on the dual-core ESP-32 system. In addition, a batching solution was implemented to alleviate memory constraints during the testing process. These optimizations were tested on two neural networks running on the ESP-32 system: a feed-forward network consisting of 8 layers and a convolutional neural network with 16 layers, both performing image classification on the MNIST database. Experimental results of multi-core execution of the CNN's neural network layers demonstrate a significant 38% reduction in runtime, but accompanied by a 34% increase in heap memory usage compared to the original model. The feed-forward network exhibited a runtime improvement only when using multi-core execution of batches, without treating layers as individual tasks, indicating that the overhead introduced by FreeRTOS tasks can sometimes outweigh the benefits of multi-core execution. Overall, this thesis provides insight into the advantages and trade-offs of optimized task execution for neural networks using FreeRTOS, laying the groundwork for more efficient task-based neural network inference on embedded systems.

2012 ACM Subject Classification Real-time operating systems, Embedded systems, Neural networks

Keywords and phrases Neural network Inference, FreeRTOS, Embedded systems

Acknowledgments

I would like to express my sincere gratitude to my reviewers, Prof. Dr. Jian-Jia Chen & Dr.-Ing. Christian Hakert, for providing me with the topic for this thesis and for their guidance and feedback throughout the course of this work.

I would also like to thank Sebastian Buschjäger, who not only developed an impressive tool with FastInference, but also was willing to provide helpful insights and answer questions about its use.

As this thesis and the completion of my bachelor's degree mark a significant milestone in my life, I want to thank my friends and family for always supporting me. I want to especially mention my best friends Fabian, Luca, Sven & Serena for their constant support and all the wonderful moments we have shared. I also want to thank Muhammed Büyük & Arthur Werlang, who accompanied me through many modules, intense Uniliga matches, and with whom it was always fun to study for the exams. I would like to express my heartfelt thanks to Laura for giving me strength and bringing joy during the final weeks of this thesis. The moments of laughter we shared made this challenging period much more manageable and memorable.

Finally, I want to thank my parents Jörg & Kirstin, as well as my brother Oliver, for their unwavering support throughout my degree journey.



Contents

1	Introduction	5
1.1	Motivation	5
1.2	Goals	5
1.3	Structure	5
2	Related Works	6
3	Fundamentals	7
3.1	Neural networks	7
3.1.1	Structure of neural networks	7
3.1.2	Training of neural networks	8
3.1.3	Types of neural networks	9
3.2	RTOS	12
3.3	FreeRTOS	12
3.3.1	Task management	12
3.3.2	Scheduling algorithms	16
3.4	Multithreading on embedded systems	19
4	Implementation	20
4.1	Discussion of original models	21
4.2	Code structure overview	22
4.3	Implementation of individual tasks	23
4.4	Task Synchronization	25
4.5	Batching Implementation	26
4.6	Multithreading with FreeRTOS	28
4.6.1	Multithreading by batching	28
4.6.2	Multithreading in layers	29
5	Evaluation	31
5.1	Performance & memory benchmarks	31
5.1.1	Task-based implementations	31
5.1.2	Batching	32
5.1.3	Multithreading – Batching and Layers	33
5.1.4	Concurrent Execution of two separate models	35
5.2	Evaluating the Pros and Cons of FreeRTOS Tasks Implementations	36
6	Conclusion	38
6.1	Summary	38
6.2	Outlook for future works	38
A	Appendix	40
A.1	Difficulties	40
A.2	Declaration of AI usage	40
A.3	List of Figures	41
A.4	List of Listings	41

1 Introduction

1.1 Motivation

Embedded systems are increasingly incorporating machine learning techniques, particularly neural networks to enhance automation and decision-making capabilities. For instance, in autonomous vehicles, embedded systems leverage neural networks to perform critical tasks such as object detection, path planning, and real-time decision-making, which are essential for safe navigation and obstacle avoidance.

The deployment of neural network models on these resource-limited systems remains a challenge, especially due to constraints in memory, processing power, core count and real-time performance requirements. To mitigate these limitations, FreeRTOS can serve as a efficient solution by providing a lightweight, real-time operating system capable of management and execution of tasks. By employing diverse task modeling strategies that align with hardware capabilities, such as multi-core execution, the execution of neural networks can be optimized within the runtime environment. However, the limited memory resources available in embedded systems need to be considered, as multi-core execution tends to increase memory usage.

1.2 Goals

This thesis examines the execution of neural networks as FreeRTOS tasks, focusing on optimizing task management, memory usage, and performance through techniques such as task synchronization, batching and multithreading. One goal is to analyze the execution of neural network layers as FreeRTOS tasks and compare the original models to the FreeRTOS variants in terms of execution time and memory usage. Therefore, highlighting the advantages and trade-offs associated with the use of FreeRTOS tasks in a neural network context.

1.3 Structure

This thesis is structured into 6 chapters. After this introductory chapter, chapter 2 gives an overview over related works, highlighting advancements and research efforts in the same space. chapter 3 gives an overview of the theoretical basics of neural networks and their training process, the key concept of a RTOS, FreeRTOS tasks and how to employ them and multithreading in context of embedded systems. Chapter 4 outlines the implementation and synchronization of neural network layers as FreeRTOS tasks and also highlighting the implementation of batching and multithreading. In chapter 5 we will examine the benchmark results generated from the implementations in the previous chapter, including performance and memory comparisons between the original neural network models and the task-based implementations, as well as the impact of batching, two different multithreading approaches, and concurrent execution of two neural network models. Finally chapter 6 concludes this thesis by providing a summary of the findings and suggestions for future works.

2 Related Works

In recent years the integration of neural networks into embedded systems has become an active area of research, driven by the need for low-power and cost-effective solutions that can perform complex tasks such as image recognition and classification.

One notable study that is similar to this thesis is "Research of the Characteristics of a Convolutional Neural Network on the ESP32-CAM Microcontroller". [1] The study highlights the performance and accuracy differences when running the same Convolutional Neural Network on a laptop versus on a microcontroller, specifically the ESP32-CAM. While this study primarily examines neural network execution in embedded systems, it does not incorporate FreeRTOS.

The next study, however, integrates FreeRTOS into its approach and is titled "Low-Power Embedded System for Gait Classification Using Neural Networks".[2] This research addresses the challenge of detecting abnormal foot postures during gait by leveraging an embedded system with a neural network. FreeRTOS is utilized by the microcontroller to manage tasks such as acquiring sensor data, communicating bidirectionally with a Bluetooth module, performing neural network classification and executing periodic sensor readings. The layout and functionality of the tasks are presented in a figure, but unfortunately, the study does not provide the implementation details or the source code.

The optimization of neural networks for embedded systems is also an active and rapidly advancing field of research, focused on enhancing model efficiency, reducing execution time, and minimizing energy consumption. Researchers are developing techniques such as model compression, quantization, and hardware-aware adaptation to address the unique constraints of embedded devices. In [3] the authors present a framework for optimizing neural network models in embedded systems, by analyzing the neural network to identify factors contributing to high execution times. They then apply compression algorithms to adjust the network structure, resulting in reduced execution time and energy consumption. Another recent example is [4], where the authors present also a framework for optimizing and deploying neural networks on embedded systems. This is accomplished through layer pruning, an optimization technique that removes structures with redundant or minimal impact, and also with quantizing the neural network model to 8-bit, therefore freeing up memory on the embedded system.

Since this thesis also addresses scheduling, another relevant development in the field is the increasing emphasis on energy-aware scheduling. Recent research has focused on optimizing task execution in embedded and heterogeneous multiprocessor systems under energy constraints. For instance, one study tackles the scheduling of dependent tasks on heterogeneous processors by proposing an energy-aware algorithm that models task dependencies and strategically allocates energy resources to optimize performance.[5]

3 Fundamentals

This chapter centers around the core principles and technologies necessary for understanding the following chapters. Section 3.1 introduces the fundamental structure and training process of neural networks. After that, section 3.2 explores the characteristics of a RTOS. Section 3.3 then gives an overview over how FreeRTOS tasks are effectively employed and delves into the details of how the scheduler manages task scheduling. Lastly, section 3.4 examines multithreading in context of embedded systems.

3.1 Neural networks

Artificial neural networks, often referred to as neural networks, are computing systems that are inspired by the human brain and designed to recognize patterns and solve complex problems.[6] They are the foundation of deep learning, which has revolutionized fields like image recognition, language models and robotics.

This section begins with the structure of a neural network, where we will examine the individual parts of a neural network and how they operate. After that, the next topic is the learning and training of a neural network. Lastly, we look upon a few different types of neural networks and highlight their features.

3.1.1 Structure of neural networks

Neural networks are made up of many interconnected nodes, also called *neurons*, that process input data and send the result to the next neuron. The neurons process the input data, which most of the times comes from other neurons, by summing up the input values and then passing this value through an *activation function*. [6] A activation function transforms the input data non-linearly, which enables the neural network to learn complex relationships in the input data. Common activation functions include the sigmoid activation function (eq. 1) which squashes the input into a range between 0 and 1.[7]

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (1)$$

The sigmoid activation function is often used for classification problems, where the neural network is trained to classify an image. For example, the output of the function can be interpreted such that any output under 0.5 represents class A, while any output greater than 0.5 represents class B.[8]

Neurons are interconnected via *synaptic weights*, often referred to simply as *weights*. [6] These are representative for the strength of a synapse, which are the neural contacts between nerve cells in the human brain. When one neuron sends its result to the next neuron, it is multiplied by the corresponding weight of that connection. These weights can be either positive and amplify the input value or negative and diminish the input value.[8] This can

be expressed by the following formula:

$$z_j = \sum_{i=1}^n x_i \cdot w_{ij} + b_j \quad (2)$$

where:

z_j = weighted sum for the j -th neuron

x_i = input value of the i -th input

w_{ij} = weight connecting the i -th input to the j -th neuron

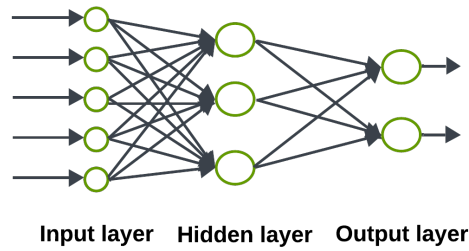
b_j = bias term for the j -th neuron

n = number of inputs to the neuron

The *bias* allows the neural network to shift the activation function, which helps when the weighted sum of the inputs is zero.

Neural networks are organized into layers, where each layer serves a distinct role in processing the input data. A typical architecture of a neural network consists of three types of layers: the *input layer*, one or more *hidden layers* and the *output layer*. All layers contain neurons and these are connected via weights to the neurons of the previous and next layer. The input layer receives raw input data, for example in case of image classification it would receive pixel values of an image. There is being no computation done in the input layer. The data of the input layer is being passed to the hidden layers, where the computation and learning of the network resides. This layered structure allows the network to progressively extract more features and then make an accurate decision. In case of a image classification problem, the first hidden layer might detect edges, while deeper hidden layers detect shapes and eventually objects we try to classify. At last, the output layer produces the final prediction or classification.[6]

The connections between neurons are used to determine the primary type of neural network architecture. If the output of a neuron is being sent to one of the previous neurons as input, we call the network a *recurrent neural network*. If no outputs are sent back as inputs anywhere in the network, then we call the network a *feed-forward neural network*. [6] The figure 1 down below gives an idea how a simple multi-layer feed forward neural network can be structured.



■ **Figure 1** A multi-layer feed-forward neural network structure[6]

3.1.2 Training of neural networks

The key feature of a neural network is the ability to learn and improve its accuracy. Training a neural network is an iterative process in which *training data* with known classifications

is presented sequentially. At the beginning of this process the neural network does a lot of errors, but with each iteration it will make small changes to the weights and will increase the accuracy over time. The training can be implemented using various learning algorithms, one of which is called *back-propagation*, commonly used for feed-forward networks.

The first step of back-propagation is to pass the input data through the network and then compute the *loss*, so how far the result deviated from the expected output. Once the overall loss is computed of the whole network, the loss is propagated backward starting from the output layer through the network. At each layer, the network now calculates how much each neuron contributed to the overall loss, allowing for adjustments to the connected weights. For every weight a gradient is being calculated, which represents the sensitivity of the loss to a small change in the weight. When for every weight a gradient is computed, the weights are updated using a optimization technique called *gradient descent*. In this technique, the weights are adjusted to the minimization of the overall loss, where a parameter called the learning rate determines how large the step of the new weight should be. If the learning rate is too large, the model might overshoot over the optimal solution, but if it is too small, it might take too long to converge or get stuck in a local minima. The calculation of the new weight is presented in equation number 3, where w_{old} is the old weight, η the learning rate and $\frac{\partial L}{\partial w}$ the gradient of the loss with respect to the weight.

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w} \quad (3)$$

This process of inputting data, calculating the loss and adjusting the weights is repeated multiple times over the training data. Over time, as the weights are adjusted, the neural network reduces its loss and increases its accuracy. This is repeated, until the loss no longer changes, indicating that the network has converged on a solution. [6, 8]

After the neural network has been trained, the *reference accuracy* describes how accurately the network makes correct predictions on average. This can be tested with *test data*, which is different from the training data used in the training process. It is important that these are separated, in order to test the neural network's generalization ability, which is its ability to accurately classify input data it has not encountered during training.[8]

The training process of a neural network can be summarized as follows: Initially, the weights are assigned random values. Then, the iterative process of back-propagation begins. For each piece of training data, the network computes the output, calculates the loss, and determines the gradients for each weight. The loss is then back-propagated through the network, and the weights are adjusted accordingly. This cycle repeats until the loss is significantly reduced and stabilizes, indicating that further iterations do not lead to meaningful changes. At this point, the training process is finished and the network can be tested using test data.

3.1.3 Types of neural networks

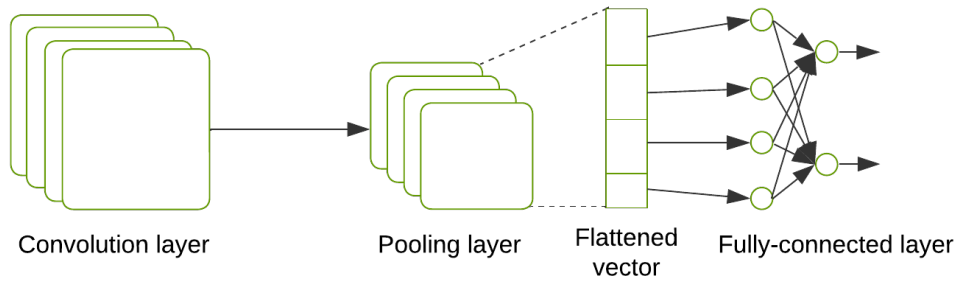
After we examined how a neural network is trained, we now dive deeper into a few different types of neural networks. From networks designed to process visual information to those optimized for embedded systems with a low amount of memory. In this sub-section, we explore several key types of neural networks, including Convolutional Neural Networks and Binarized Neural Networks for embedded systems.

3.1.3.1 Convolutional neural networks

In the previous section we had the example of a neural network that purpose it was to classify objects in a given image. The architecture used was a *multi-layer Perceptron* (MLP), which

is able to solve problems that not circumvent around classifying images but also for example speech recognition. As previously explained, a MLP can classify images, but it is limited to handling only small-sized images. A common dataset used to benchmark neural networks is the MNIST database, which contains 28x28 images of handwritten digits. When using a MLP, a neuron in the first hidden layer would have 784 weights attached to it, because every pixel would be a separate input in the input layer. 784 weights are still manageable, but this does not scale well to larger images. With an example size of 64x64 every neuron in the first hidden layer would have 4096. Moreover, the images of the MNIST database are normalised to black and white values. If the neural network has to also consider the color of pixels in the computation, we would need 12,228 weights for each neuron. The reason is that each pixel now requires three values for the representation of color: one red, one green and one blue value. This rapid increase in the number of weights does not only make the training process slow, it also increases the risk of overfitting.[9] *Overfitting* occurs when a neural network is trained with a vast amount of parameters and insufficient training data, causing it to memorize the training data rather than generalize from it.[8] This leads to poor accuracy when the neural network is tested with test data.

This is where Convolutional Neural Networks (CNNs) become essential, as they are specifically designed to handle high-dimensional image data efficiently. A CNN has the following layers: input layer, convolutional layer, pooling layer and fully-connected layer.

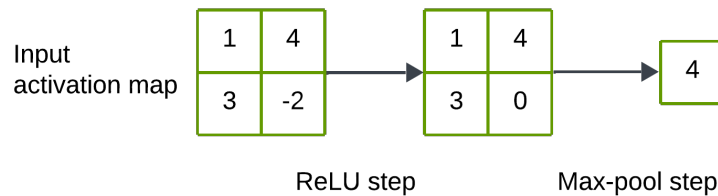


■ **Figure 2** Example of a simple CNN architecture containing only the key layers

The *convolutional layer* is the core building block of a CNN. Instead of connecting each neuron to every pixel, the convolutional layer applies small, learnable filters to small regions of the input image. These filters have a typical size of 3x3 and slide across the entire image to detect their specific feature. A feature can be vertical edges, or textures for example. After each filter has completed the entire image, an *activation map* is returned for each kernel, indicating where the specific feature detected by the kernel appears in the image. After the activation maps have been computed, a *Rectified Linear Unit*(ReLU) is commonly applied to them. The ReLU activation function works by setting all negative values in the activation map to zero, while leaving positive values unchanged. This introduces non-linearity into the network, which is essential for learning complex features and patterns in the data. By transforming the activation maps in this way, ReLU allows the model to focus only on the most relevant features, effectively filtering out non-essential or negative signals.[9]

After the convolutional layer and the use of ReLU on the activation maps, the *pooling layer* aims at reducing the height and width of the activation maps, while retaining the most important information. The most commonly used method is the *max-pooling*, where, in a

2x2 area of the activation map, only the largest value is kept and written in the new, smaller activation map.[9] Figure 3 presents an example ReLU activation and a max-pooling for a simplified 2x2 activation map.



■ **Figure 3** Example of ReLU activation followed by max-pooling for a 2x2 activation map

Another type of layer commonly used after convolutional layers, or more generally after a neural network has performed some computations, is the *Batch Normalization layer*. The Batch Normalization layer normalizes the activations of the previous layer by adjusting and scaling them to make the training more efficient. This normalization technique has proven to accelerate convergence during training by enabling larger learning rates and reducing issues that could cause training to fail.[10]

As the network progresses through several convolutional, ReLU, and pooling layers, the height and width of the image are reduced, but the number of activation maps typically increases, allowing the network to capture a variety of important patterns. Eventually, the reduced activation maps are flattened into a single vector, which serves as the input to the next stage: the *fully-connected layer*, also called general matrix multiply or *Gemm layer*. In the fully-connected layer, each neuron is connected to every neuron from the previous layer. This is similar to the architecture used in a traditional MLP. The purpose of a fully-connected layer is to perform feature extraction and transformation. Most of the time, it increases or decreases the amount of neurons used, in comparison to the previous layer. The final layer of a neural network for classification can be a softmax, or also called a *LogSoftmax layer*, which transforms the output of the neural network into a set of logarithmic probabilities for each class. For instance, if the neural network is designed to classify an image into one of three classes, the LogSoftmax layer will compute the likelihood for each class, adding up to 1, indicating how likely the image belongs to each of the three categories.[11] Through this combination of feature extraction and decision-making, CNNs can effectively handle complex tasks such as image classification and object detection. [9]

3.1.3.2 Binarized neural networks

Binarized Neural Networks (BNNs) are a type of neural network where the weights and activation functions are constrained to binary values, typically -1 and +1, instead of full precision values like floating points that are typically 32-bit. This reduces the computational complexity and memory requirements significantly, making BNNs highly suitable for resource-constrained environments, such as embedded systems.[12]

The binarization process simplifies the arithmetic operations in the neural network. Instead of using costly multiplication and addition operations, BNNs replace them with efficient bitwise operations such as XNOR and bit-counting.[13]

The key advantage of BNNs is their ability to perform well on hardware with limited resources, which is achieved by reducing the neural network model size and computational

cost. However, the simplification that comes with binarization is not without trade-offs. The most notable challenge of BNNs is the loss in accuracy that comes with reducing weights and activation functions from 32-bit floating point to 1-bit binary values. Researchers try to mitigate this issue by using for example partial binarization, where only a part of the network is binarized.[12]

3.2 RTOS

A *Real-time operating system* (RTOS) is a specialized operating system designed for applications that must meet time constraints, such as threads that must be completed within a specific time window, often in milliseconds. In comparison, a desktop operating system like Windows prioritizes keeping the computer responsive for the user, while the time constraints of threads are not as critical. While a RTOS prioritizes the time constraints, it also needs to be deterministic so the scheduling is being done in a predictable manner.[14] RTOSs' can also be further categorized into a *hard RTOS* and a *soft RTOS*, which describes the consequences in case a deadline of a task gets missed. In case of a hard RTOS, the consequence of missing a deadline of a task could be catastrophic. Examples for a hard RTOS could be an airbag system in automobiles, where if the airbag fails to deploy within the required time of a collision, it could result in severe injury or death. In contrast, a soft RTOS can tolerate a delay, but it is still undesired.[15] Interestingly, FreeRTOS is capable of functioning as either a soft or hard RTOS, due the pure priority scheduling system it utilizes, which we will investigate in the next section.

Overall, a RTOS is an operating system designed with a focus on deterministic, time-limited scheduling. While an RTOS also manages functions such as interrupt management, memory allocation, and device control, in this thesis we will only focus on scheduling and resource management, with a deeper dive into how these aspects are handled by FreeRTOS in the next section.

3.3 FreeRTOS

FreeRTOS was originally developed by Richard Barry around 2003 and was later further developed and maintained by his own company Real Time Engineers Ltd. In the year 2017, Real Time Engineers Ltd. passed the project to Amazon Web Services (AWS) who now develop FreeRTOS. Richard Barry still works on FreeRTOS as part of the AWS team. FreeRTOS is a real-time kernel designed for building embedded applications that require strict real-time performance. It enables applications to be executed as a set of independent execution threads, allowing efficient thread management and therefore ensuring that the required real-time performance is being constantly achieved. These execution threads are called *tasks* in FreeRTOS.

We begin this section by first looking into how to create and manage tasks. After that we examine how the FreeRTOS scheduler operates and what different scheduling options FreeRTOS offers. The main source for this entire section is the official FreeRTOS documentation written by Richard Barry with the title: Mastering the FreeRTOS Real Time Kernel.[16]

3.3.1 Task management

This sub-section provides an in-depth look at the task lifecycle, including the creation of tasks, their execution and coordination using synchronization mechanisms such as semaphores, mutexes, and task notifications, and finally, their deletion.

3.3.1.1 Task Creation

In FreeRTOS, tasks are created by calling task functions, which are standard c functions. The only requirement for a function to be executed as a task, is that it must have a return type of void and take a void pointer as parameter. This allows flexibility for the passing of data types to the task.

■ Listing 1 Example task function[16]

```
void ATaskFunction( void *pvParameters );
```

Now that we have a function that we can call, we can create our task which will be added to the schedulers list of tasks for execution, with the `xTaskCreate()` function.

■ Listing 2 xTaskCreate function[16]

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,
                        const char * const pcName,
                        uint16_t usStackDepth,
                        void *pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t * pxCreatedTask );
```

1. **pvTaskCode** (`TaskFunction_t`) A pointer to the task function that we desire to call.
2. **pcName** (`const char * const`) A human-readable name for the task. This is optional and does not get used from FreeRTOS in any way, but it is useful for debugging.
3. **usStackDepth** (`uint16_t`) Each Task has its own stack which is allocated by the FreeRTOS kernel when the task is created. This parameter specifies how large the stack for this task should be, but not in the number of bytes, instead in the number of words. The size of a word depends on the architecture FreeRTOS runs on, if for example the stack is 32-bit wide, one word would be 4 bytes long. If now 100 is passed for **usStackDepth**, the kernel would allocate 400 bytes for the stack.
It is important for the task to have enough stack space to execute its functions. There is however, no easy way to calculate the right amount of stack space required for a task to execute without a stack overflow occurring and also not to allocate too much memory.
4. **pvParameters** (`void`) A pointer to any parameters that should be passed into the task. This is also optional and can be set to NULL.
5. **uxPriority** (`UBaseType_t`) This value defines the priority at which the task will be managed from the scheduler. The lowest value is 0, which represents the lowest priority. The highest priority is being represented by `configMAX_PRIORITIES - 1`. The maximum priority can be set by the user by changing the `configMAX_PRIORITIES` value.
6. **pxCreatedTask** (`TaskHandle_t`) A pointer to a handle for the task. This can be used for API calls affecting the task, for example notifying or deleting the task. This is also optional and can be set to NULL if not needed.

With the task creation function explained, it is important to understand how tasks interact with each other and how to coordinate them. After calling the `xTaskCreate` function, the system does not wait on the created task to complete. Instead the execution continues asynchronously, allowing the creation of new tasks or running code in parallel. Tasks begin to be executed after starting the scheduler using `vTaskStartScheduler()`, which assumes control over task execution and prioritization. This process will be further examined in subsection 3.3.2 later in this section.

Usually, the execution of the program should first terminate after all tasks have concluded.

To achieve this, FreeRTOS provides various synchronization mechanisms like semaphores, mutexes and task notifications. Up next, we will explore how to implement these synchronization mechanisms and discuss the most effective scenarios for using each one.[16]

3.3.1.2 Task Synchronization via semaphores & mutexes

Reliable and sufficient inter Task mechanisms are one feature deemed crucial for developing with a RTOS.[15] The tasks must communicate and coordinate with each other to avoid conflicts while accessing common data, but also to notify the main method when the execution is finished. We will first examine semaphores and mutexes as synchronization mechanisms, followed by task notifications. FreeRTOS supports two types of semaphores: binary and counting semaphores. The *binary semaphore* can only take two values, 0 and 1 and is primarily used for signaling between tasks or as a simple flag, that indicates that a task finished its execution. The *counting semaphore* on the other hand can count up to a specified value and is more versatile. For example it can be used to signal to a function that two tasks it started have completed, once both tasks have incremented the value to 2. FreeRTOS also offers to use mutexes, which similar to the binary semaphore can only have two states: locked or unlocked. A mutex and binary semaphore may appear similar, but they serve different purposes and have contrasting behaviors.

The first difference is ownership. Semaphores do not have ownership, so no task owns the semaphore if they lock it. If a task takes a binary semaphore, thereby locking it, another task can give the semaphore and unlock it again. Mutexes do have ownership and if a task locks the mutex, this specific task has to unlock it.

Another difference is the handling of *priority inversion*, which is a problem that can arise if a higher prioritized task needs to wait on a mutex that is being blocked by a lower prioritized task. Priority inversion can become a bigger issue, if an intermediate prioritized task, between the lower and higher prioritized task, interrupts the lower prioritized task before it could release the mutex, further delaying the higher prioritized task. This problem gets solved for mutexes by including a *priority inheritance* mechanism. With priority inheritance the lower prioritized task is temporarily being raised to the same priority of the higher prioritized task waiting on the mutex. This prevents intermediate prioritized tasks from interrupting the lower prioritized task and therefore minimizing the time where the higher prioritized task is delayed. After the lower prioritized mutex unlocks the mutex, its priority is reset to the original value. Priority inheritance is exclusive to mutexes. Semaphores do not have this functionality, which is derived from their purpose of being used as signaling between tasks and not as managing exclusive access to one resource. Moreover, the lack of ownership for semaphores eliminates the possibility that one task can only free the access another task is waiting on. The purpose of a mutex is not to be used for communication between tasks, but rather to manage the exclusive access to a shared resource, ensuring that only one task can use it at a time.

While the purposes and behaviors of binary semaphores and mutexes are different, the structure of the function to create either one are identical.

■ Listing 3 Binary mutex creation function[16]

```
SemaphoreHandle_t xSemaphoreCreateBinary( void );
```

■ Listing 4 Mutex creation function[16]

```
SemaphoreHandle_t xSemaphoreCreateMutex( void );
```

The function to create a counting semaphore will take two parameters, one for the maximum value it can count up to and one to set the initial counting value.

■ **Listing 5** Counting semaphore creation function[16]

```
SemaphoreHandle_t xSemaphoreCreateCounting( UBaseType_t uxMaxCount,
                                             UBaseType_t uxInitialCount );
```

All functions return a SemaphoreHandle in which, in case of a successful creation, the semaphore or mutex is being stored in. If NULL is returned, the heap memory was not sufficient to allocate the semaphore or mutex.

The usage of both semaphores and mutex is also identical, with all three utilizing the same function for increasing or decreasing its value.

■ **Listing 6** Semaphore take function[16]

```
BaseType_t xSemaphoreTake( SemaphoreHandle_t xSemaphore,
                           TickType_t xTicksToWait );
```

The parameters are the semaphore or mutex we want to take from and with `xTicksToWait` the amount of time the task should wait, if the semaphore or mutex is not already available. If the value of `xTicksToWait` is set to 0, the function will directly return if the semaphore or mutex is not available. In contrast, `portMAX_DELAY` will cause the task to wait indefinitely for the semaphore or mutex to become available. The return value is either `pdPASS`, when successfully decreasing the value, or `pdFALSE` if the semaphore or mutex is not available and the time specified in `xTicksToWait` ran out.

The function to increase the value of a semaphore or mutex requires only one parameter, namely the SemaphoreHandle which value should be increased.

■ **Listing 7** Semaphore give function[16]

```
BaseType_t xSemaphoreGive( SemaphoreHandle_t xSemaphore );
```

The possible return values are again `pdPASS`, if the increment was successful and `pdFALSE` if an error occurred while increasing the value.

Lastly, the SemaphoreHandle which holds the semaphore or mutex, can be deleted with the `vSemaphoreDelete` function, freeing up the heap memory allocated for it.[16]

■ **Listing 8** Semaphore delete function[16]

```
void vSemaphoreDelete( SemaphoreHandle_t xSemaphore );
```

3.3.1.3 Task synchronization via Notifications

Task notifications are a lightweight and efficient method of communication and synchronization between tasks. When using a semaphore or mutex in FreeRTOS, the tasks communicate with each other via a communication object, in this case the semaphore or mutex. Task notifications allow the tasks to directly communicate with each other, cutting out the need for a communication object. This yields performance and memory benefits, promising to be faster and also consume less memory than using a semaphore. The memory overhead for using task notifications are merely 8 bytes of memory per task. However, these benefits come with a few limitations, as there are scenarios where task notifications can not be used. Two examples would be not being able to buffer multiple data items, as can be done with a counting semaphore, and not being able to broadcast to multiple tasks simultaneously. This can be

done with another communication object, the event group, which is not further examined in this thesis. With task notifications, there is no need for calling a creation function, as each task has an array of task notifications already. The usage of task notifications is similar to the usage of a semaphore or mutex, with similar methods for giving and receiving notifications.

■ **Listing 9** Task Notification give method[16]

```
BaseType_t xTaskNotifyGive( TaskHandle_t xTaskToNotify );
```

The function for notifying only expects the task handle for the recipient. The return value possible for this function is only `pdPASS`, because `xTaskNotifyGive` is essentially only a macro that calls a more flexible function called `xTaskNotify`. This more complex function gives the ability to not only increment the notification value, but also to update a task's notification state and overwrite the notification value. This function extends beyond the scope of this work, as the functionality of `xTaskNotifyGive` is sufficient for our context. The function to take a notification is called `ulTaskNotifyTake` and expects two parameters. One of them is the `xTicksToWait` parameter, which is equivalent to the one used for the `xSemaphoreTake` function. The other parameter is `xClearCountOnExit` which can be set to either `pdTRUE` or `pdFALSE`. If set to `pdTRUE`, the task notification value of the receiving task will be cleared to zero before the `ulTaskNotifyTake` returns. If the value is set to `pdFALSE`, it decrements the task notification value before the function returns, if value is greater than zero. The task notification value of the calling task is also the returned value of this function, before it was adjusted by the `xClearCountOnExit` parameter.

■ **Listing 10** Task Notification take function[16]

```
uint32_t ulTaskNotifyTake( BaseType_t xClearCountOnExit,
                          TickType_t xTicksToWait );
```

Overall, task Notifications are a efficient and lightweight tool to communicate between tasks, but it can not replace communication objects in all scenarios.[16]

3.3.1.4 Task deletion

After the task has successfully run and notified other tasks that they can proceed, it is important to delete the task. This is especially critical in embedded systems due to the limited memory. The deletion frees up heap memory, which in a larger neural network is in high demand.

■ **Listing 11** Task delete function[16]

```
void vTaskDelete( TaskHandle_t pxTaskToDelete );
```

The function takes only one parameter, which can be the handle to the task that needs to be deleted. This can also be set to `NULL`, when the task should delete itself.

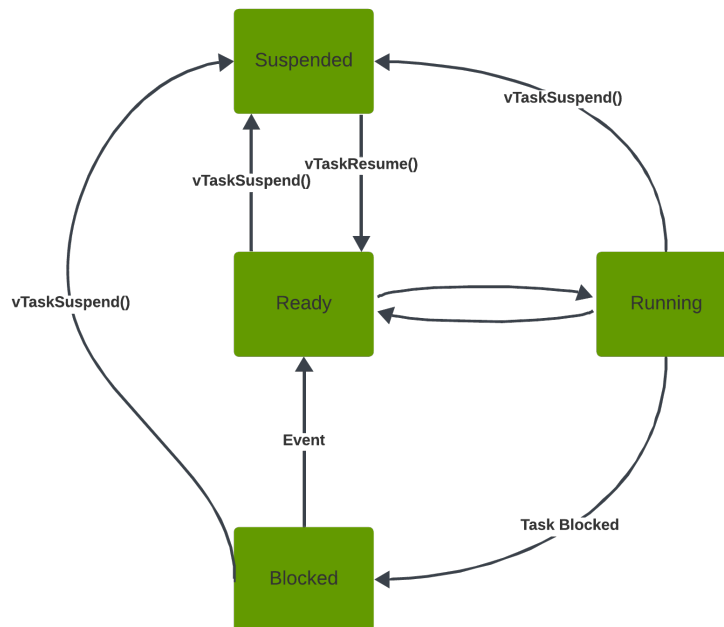
3.3.2 Scheduling algorithms

This subsection is centered around the different scheduling algorithms that can be used in FreeRTOS and how they function. To better understand this we must dive a little deeper and examine what task states are and how a task can transition between them.

3.3.2.1 Task states

In FreeRTOS, tasks can exist with one of four states: *Running*, *Ready*, *Blocked* and *Suspended*. A task is in the state *Running*, when it is currently executing on one of the CPU cores. In a system with only one CPU core, there can only be one task executing at the same time, so only one task can have the state *Running* at all times. From the *Running* state, the task switches into one of the other three states, which can be summarized into "Not Running" states. In the *Ready* state, the task is prepared to run, but needs to wait on the scheduler to allocate CPU time for it. The task is not hindered by external conditions, only by CPU availability. Tasks enter the *Blocked* state, when waiting for a specific event. This event can be either a temporal event or a synchronization event. A temporal event can be a delay period expiring, for example when taking a semaphore and setting the `xTicksToWait` parameter to a specific value. Synchronization events originate from the synchronization tools we already examined like semaphores, mutexes and task notifications. If a task is trying to claim a mutex that is already locked, it enters the *Blocked* state, because of this synchronization event. When the mutex is then later unlocked, the task switches from the *Blocked* into the *Ready* state. Lastly, in the *Suspended* state a task is paused and cannot run until being called by the `vTaskResume()` method. A task can be suspended in all other states by calling the `vTaskSuspend()` method on it. Unlike in the *Blocked* state, the tasks in the *Suspended* state are not hindered by external conditions, but manually suspended from running.

Figure 4 visually represents the four task states and transitions between them.[16]



■ **Figure 4** Task States transitions[16]

3.3.2.2 Prioritized pre-emptive scheduling

The first scheduling algorithm we examine is fixed priority pre-emptive scheduling with time slicing, which is also the default scheduling algorithm used by FreeRTOS.

With this algorithm tasks are assigned a fixed priority and can only be temporarily raised due to priority inheritance by the scheduler. This does not prevent tasks from lowering or raising priority themselves, but only that the scheduler does not change the priorities. The task with the highest priority in the Ready state is always selected to transition into the Running state for execution. In pre-emptive scheduling, if a task in the Ready state has a higher priority than the currently executing one in the Running state, the executing task gets immediately interrupted and moved into the Ready state. The higher prioritized task then takes over and enters the Running state. This ensures that the scheduler always gives CPU time to the most critical tasks first.

Lastly, time slicing ensures that tasks of the same priority are executed an equal amount of time, called a time slice, before the next task gets the equal time slice for its execution. This allows tasks of the same priority to execute in a fair and balanced way. The tasks continue to execute and switch in this round-robin manner until a higher priority task enters the Ready state. In which case the currently running task is preempted and the higher priority task executes. The higher priority task does not have the constraint of a time slice, it runs until it is either blocked, finished or suspended.

FreeRTOS offers the option to disable time slicing, therefore reducing the amount of context switches and schedulers overhead, because the tasks of same priority do not need to switch constantly. That being said, this can lead to tasks of equal priority receiving vastly different amount of execution time. Therefore turning off time slicing is only recommended to be used by experienced users.[16]

3.3.2.3 Cooperative scheduling

FreeRTOS also offers a different algorithm than pre-emptive scheduling, where the tasks do not get preempted by the scheduler when a higher prioritized task is available. This algorithm is called cooperative scheduling and relies on the voluntarily yielding of the CPU time from tasks in the Running state. The scheduler will still select the highest prioritized task from the Ready state, if there is currently no task in the Running state. If there is a task in the Running state and a higher prioritized task enters the Ready state, the executing task will not get preempted and continues executing. A task can only switch out of the Running state if entering the Blocked State, waiting for an event or resource, or by calling `taskYIELD()` and switching into the Ready State. Time slicing is not utilized in cooperative scheduling, because tasks are never preempted and therefore can not be forced to switch back into the Ready state.

This scheduling method requires the user to carefully design the tasks to avoid that any task monopolizes the CPU. Also the cooperative scheduling makes the system less responsive, which can be crucial in real-time systems where critical high prioritized tasks can arise and need immediate execution time. [16]

In our context of neural networks in embedded systems, the usage of cooperative scheduling seems to be beneficial. The execution of neural networks and their layers are often sequential, where each layer process input data before passing it to the next layer. This does align well with cooperative scheduling, as each layer can be designed to yield control after completing its computations.

3.4 Multithreading on embedded systems

Multithreading refers to the ability of a processor or to execute multiple threads concurrently, allowing for parallel execution of tasks. Today's embedded systems often come with more than one CPU core, which allows an embedded system for example to handle sensor input, perform calculations and manage communication simultaneously. In our context of neural network execution, this approach can greatly reduce the network's runtime, as we will see in chapter 5.

In embedded systems, multithreading is typically managed by a RTOS like FreeRTOS, which schedules its tasks on the available cores. FreeRTOS also offers us the ability of task core pinning, which allows tasks to be assigned to specific cores. By pinning tasks to specific cores, you can minimize the time spent switching tasks between cores and ensure that high-priority or time-critical tasks always run on the most suitable core. This helps reduce context-switching overhead and improves the predictability of task execution, which is crucial in real-time systems. It is crucial to avoid overloading a single core with too many pinned tasks, as this can lead to significant imbalance in load distribution. The function to create a task and pin it to one specific core is shown in listing 12, which has the same parameters as the `xTaskCreate` function, but with one addition. The `xCoreID` parameter takes the number of the core the task should be pinned to. In a dual-core system, 0 would be the first and 1 the second core.[16]

■ **Listing 12** Task creation pinned to one core[16]

```
BaseType_t xTaskCreatePinnedToCore( TaskFunction_t pvTaskCode,
                                     const char * const pcName,
                                     uint16_t usStackDepth,
                                     void * pvParameters,
                                     UBaseType_t uxPriority,
                                     TaskHandle_t * pvCreatedTask,
                                     const BaseType_t xCoreID );
```

Embedded systems often face constraints in terms of available resources, such as memory, power and computational power. Especially the memory constraint poses a significant challenge in case of neural networks, when trying to introduce concurrent execution of two batches, a topic we explore further in chapter 4. Also the synchronization tools we looked upon in the previous section need to be put in good use to control the different threads and prevent race conditions, deadlocks and data corruption. Additionally, unlike in general-purpose programming, embedded systems lack many higher-level features that help with resource management, error handling and debugging, which adds complexity when implementing in embedded systems. For example, OpenMP, an API used for multi-threaded parallel processing is unavailable on the ESP-32 systems, because FreeRTOS which runs on it does not support it. Instead of using OpenMP for parallelizing a for-loop, the programmer has to use FreeRTOS tasks.

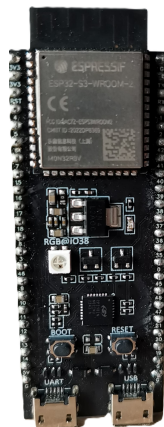
4 Implementation

This chapter dives into the implementation details of FreeRTOS tasks for neural network inference. The following sections will explore the design, implementation and execution details of the features represented in each sections title. The source code for all implementations can be found in this [Github repository](#).

The first section 4.1 introduces the selected neural network models and their layer structure. Next, section 4.2 provides an overview of the code structure and its main components. After that, section 4.3 delves into the details of implementing FreeRTOS tasks and converting layers into a task-based structure. Section 4.4 shows the three different synchronization strategies using task notifications and semaphores. This is followed up by section 4.5 which details the limitations of the initial implementation and how batching solved these issues. Finally, section 4.6 covers two multithreading approaches and explores their implementation details.

Before beginning with section 4.1, we briefly examine the platform used to implement and execute the code. The embedded system is the ESP-32-S3-DevKitC-1 from Espressif, equipped with a dual-core processor running up to 240 MHz and 512 KB of static RAM. All implementations were tested at 160 MHz, as this was the default value set by the *Espressif IoT Development Framework* (ESP-IDF). The ESP-IDF, which is the official framework from Espressif for working with the ESP systems, is used to flash code onto the ESP32 and configure FreeRTOS settings. It utilizes a custom FreeRTOS implementation which has additional features like symmetric multiprocessing, but unfortunately restricts access to the `FreeRTOSConfig.h`, which contains the values necessary for testing cooperative scheduling.[17] Without the ability to modify the required values, cooperative scheduling cannot be tested in this thesis.

One notable feature of the ESP-IDF is that the scheduler starts automatically when the user defines a `void app_main(void)` function, serving as the entry point. Therefore, in this implementation chapter of this thesis, the `vTaskStartScheduler()` function is never explicitly called, since the scheduler is already active when `app_main` begins execution.

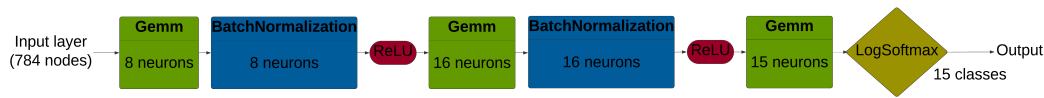


■ **Figure 5** Image of the ESP-32-S3 system

4.1 Discussion of original models

The implementation of FreeRTOS tasks involves neural network inference. Fortunately, the **FastInference** tool by Sebastian Buschjäger allows us to generate example neural networks models that we can utilize for this purpose. FastInference is a machine learning model optimizer that generates a optimal implementation of a given neural network model. It allows for various different machine learning structures as inputs, such as Decision trees or neural network models. For neural network models, it supports both feed-forward networks and CNNs, as FastInference is specifically designed to target small embedded systems with limited memory and computational power. This focus on lightweight models ensures that even with limited resources, tasks such as classification or recognition can be performed effectively without compromising performance.

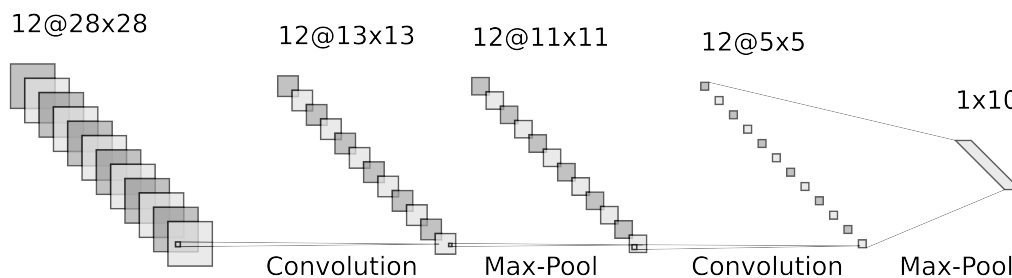
The first generated model was a simple feed-forward neural network used to classify 28x28 images of fashion items by the MNIST database.



■ **Figure 6** Architecture of the feed-forward neural network.

The input layer consists of 784 nodes, corresponding to the 28x28 images. The first layer is a Gemm layer, reducing these 784 input nodes to 8 neurons. Following this is a Batch Normalization layer, which normalizes the output from the Gemm layer. The ReLU activation function is then applied, replacing any negative values from the previous layer. This structure, consisting of the Gemm, Batch Normalization, and ReLU layers, is repeated, with the number of neurons increased to 16, and finally to 15 neurons. At the end, a LogSoftmax layer produces the final output.

The second model is a CNN also tasked to classify 28x28 images by the MNIST database. Figure 7 illustrates the largest and most essential layers of the CNN.

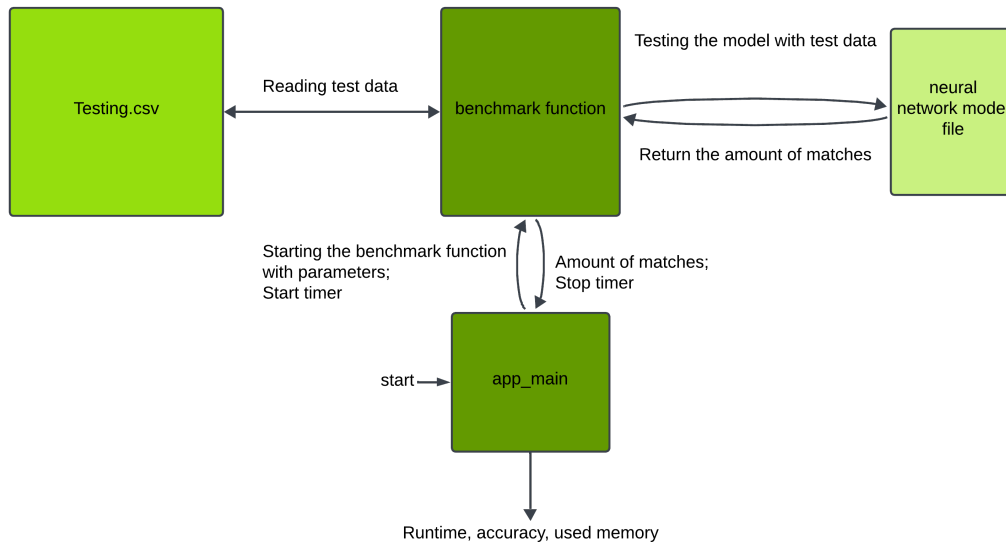


■ **Figure 7** The most important layers of the CNN. Created via the **NN-SVG** tool from Alexander LeNail.

It begins with a Reshape layer which to prepare the 28x28 image for further processing. The second layer is a typical convolution layer with filters of size 3x3 extracting features of the image. Every Convolution layer is followed by one Batch Normalization and one ReLU activation layer. Layer 5 is a max pool layer which reduces the spatial dimension of the data, shrinking the image while retaining the most important information. This structure of layers is repeated and the neural network ends with fully-connected layers and at last a LogSoftmax layer, which outputs the probability of each of the 10 classes that this model aims to classify. In total, the neural network consists of 15 layers and is considerably more computationally intensive than a simple feed-forward model. The original CNN model exceeded the limited memory capacity of the ESP-32-S3 I am using. To address this, the weights and bias values were binarized, and the number of neurons was reduced. As a result, the reference accuracy dropped from 87.9% to 70.8%.

4.2 Code structure overview

This section provides a brief overview of the program's code structure and the process of the testing sequence. Figure 8 gives an overview of the four main components of the code generated from FastInference.



■ **Figure 8** Abstract illustration of the four main components of the code and the interactions with each other

In the following sections changes will be made to three of them, except for the **Testing.csv** which holds the test data. The CSV file is flashed onto the ESP32, but only partially, as the full 10.000 lines would exceed the ESP32's limited memory capacity. Exactly 250 lines are flashed, which is sufficient for effectively testing the neural network models.

The **app_main** function serves as the entry point for the execution, where the **benchmark function** is called with parameters such as the amount of lines to test and later the batch size. Within **app_main**, the runtime measurement begins by starting a timer when the benchmark function is called. Once the function completes, the timer stops, and metrics like runtime and accuracy are printed.

The `benchmark` function first calls a function that reads data from the `Testing.csv` and stores the data in two vector objects. Next, the benchmark function calls the neural network model with these vectors and receives the count of matches. If batching is used, this process of reading, testing and receiving is repeated for every batch.

Finally, the `neural network model` file holds the weights, biases, scales and layers of the neural network model. It has one function, which is being called by the benchmark function to start the testing process. This structure enables easy swapping of the neural network model for benchmarking without needing to modify the files containing the other functions.

4.3 Implementation of individual tasks

In this section, we will explore how FreeRTOS tasks were implemented for the different layers located in the neural network model file. Our focus will be on the core aspects of task creation and the correct usage of task parameters. For clarity, the additional code related to task synchronization has been omitted here and will be discussed in detail in the following section.

To begin the implementation of the layers as tasks, we first review the original implementation of the neural network, in this case of the simple feed-forward network. Listing 13 shows the first layer, where each of the 8 neurons receives input from all 784 input nodes. For each neuron, the input nodes are multiplied by their corresponding weights and summed up to produce the neuron's output.

In case of the simple feed-forward network we generated, two additional Gemm layers will follow as later layers. Although these Gemm layers have different parameters, their loop structure remains identical. This enables us to create a generalized Gemm task, which can be called by the three different Gemm layers, with their parameters passed to it. By doing so, we reduce the amount of repetitive code.

■ **Listing 13** Original code of the gemm layer

```
// Layer 1: Gemm
for (int row = 0; row < 8; row++) {
    layer_1_output[row] = layer_1_bias[row];
}
for (int row = 0; row < 8; d++) {
    for (int column = 0; column < 784; column++) {
        layer_1_output[row] += (layer_1_weight[row][column] *
                                layer_0_output[column]);
    }
}
```

For the creation of a generalized task, we define a struct which contains all parameters needed for the specific task. Listing 14 illustrates the struct used for the Gemm task.

■ **Listing 14** Struct for the parameters when calling a Gemm task

```
struct GemmTaskParams {
    unsigned int num_rows;
    unsigned int num_columns;
    double* output;
    double* output_1;
    const double* bias;
    const double* weight;
};
```

To transform the original implementation of the Gemm layer into a FreeRTOS task, we follow the conventions of the example task function presented in the FreeRTOS section. We first set the method return type of the function to `void` and define the parameter as a `void` pointer to accept parameters of any type, like the struct for the Gemm task. Following that, we assign the parameters from the Gemm task struct to the local variables. The loop structure remains identical, but parameters are used instead of using the hard-coded values. One noteworthy exception is the calculation of the `weight_index` variable, which is necessary to select the correct value from `layer_p_weight`, a 2D array flattened into a single pointer. The GemmTask struct contains only a single pointer to this array because it has already been initialized and contains values. Additionally, the weight arrays for different layers vary in size, making it impossible to define a fixed array size within the struct. Therefore, we need to calculate an index for any multi-dimensional array we want to access inside the tasks. C and C++ follow the row-major order, where the index of an element in a multi-dimensional array is calculated as:

$$\text{2D array: } \text{index} = (\text{row} \times \text{num_columns}) + \text{column} \quad (4)$$

$$\begin{aligned} \text{3D array: } \text{index} = & (\text{depth} \times \text{num_rows} \times \text{num_columns}) \\ & + (\text{row} \times \text{num_columns}) + \text{column} \end{aligned} \quad (5)$$

■ **Listing 15** First Task implementation of gemm layer

```
void GemmTask(void *params) {
    GemmTaskParams *taskParams = static_cast<GemmTaskParams*>(params);
    unsigned int num_rows = taskParams->num_rows;
    unsigned int num_columns = taskParams->num_columns;
    double *layer_p_output = taskParams->output;
    double *layer_p_previous_output = taskParams->output_1;
    const double *layer_p_bias = taskParams->bias;
    const double *layer_p_weight = taskParams->weight;
    unsigned int weight_index = 0;
    for (int row = 0; row < num_rows; row++) {
        layer_p_output[row] = layer_p_bias[row];
    }
    for (int row = 0; row < num_rows; row++) {
        for (int column = 0; column < num_columns; column++) {
            weight_index = row * num_columns + column;
            layer_p_output[row] += (layer_p_weight[weight_index] *
                                   layer_p_previous_output[column]);
        }
    }
}
```

With the Gemm task function in place, we can create it within the `predict_SimpleMLP` function, that is executed when the neural network is assigned the task of classifying test data. In listing 16 the process of task creation is shown, where the parameters for the Gemm task are defined and then the task is created using the `xTaskCreate` function.

■ **Listing 16** Task creation example

```
void predict_SimpleMLP(double const *const x, double *pred) {
    GemmTaskParams layer1params{8,
                                784,
                                layer_1_output,
```

```

                                const_cast<double *>(layer_0_output),
                                layer_1_bias,
                                &layer_1_weight[0][0]);
    xTaskCreate(GemmTask, "Layer1", 2048, &layer1params, 1, NULL);
}

```

This procedure, of defining a generalized task function, declaring a struct which includes the needed parameters for the task, and the creation of the task for the corresponding layer, is repeated for each layer. Each layer is created one after the other with the `xTaskCreate` function. However, we must implement synchronization between these tasks, because without it, all the tasks would execute concurrently, resulting in data corruption. Since both neural networks in our context operate in a fully sequential manner, each layer must wait for the previous one to finish before executing.

4.4 Task Synchronization

In the previous section, we discussed the procedure of turning the original implementations of the layers into FreeRTOS tasks. However, creating tasks alone is not sufficient for ensuring correct execution in our neural network. Without proper synchronization, all tasks would run concurrently, leading to data corruption and incorrect results, especially in systems where data dependency exists between layers.

For the synchronization between the layer tasks we choose task notifications, because they are lightweight and specifically designed for inter-tasking signaling in FreeRTOS. To implement this, we first need to assign a Taskhandle to each task, enabling us to send notifications to the task when using the corresponding handle. For our first layer example from the previous subsection, this is illustrated in Listing 17.

■ Listing 17 Task creation with Taskhandle

```

void predict_SimpleMLP(double const *const x, double *pred) {
    xTaskCreate(GemmTask, "Layer1", 2048,
                &layer1params, 1, &layer1handle);
}

```

With each task now linked to a unique Taskhandle, we can implement the notification structure within the tasks, as illustrated in 18. The `currentTaskHandle` variable identifies the Taskhandle corresponding to the current task, indicating which layer is currently executing. Since tasks are created sequentially and would immediately begin computation, this could lead to data corruption without proper synchronization. To prevent this, we distinguish between the first Gemm layer and the subsequent layers. If the task is called with the parameters for the first layer, it is delayed for 1 tick to allow initialization of all TaskHandles. Without this delay, the execution of the neural network would irregularly throw an error. If the task is created with parameters for a later layer, it waits for a notification to its Taskhandle with the function `ulTaskNotifyTake`. With `portMAX_DELAY` set, these tasks remain in the blocked state indefinitely and will not be selected by the scheduler for execution, until they receive a notification.

Once the first layer's delay has passed or a notification has been received for the subsequent layers, the task continues like in Listing 15 to execute its computations for the input parameters. In Listing 18, the core code from Listing 15 has been omitted with `...` to focus on the synchronization structure. After the layer has finished its computations, it sends a notification with the `xTaskNotifyGive` function to the next layer's task, signaling it to begin execution. Finally, the task deletes itself with `vTaskDelete(NULL)` to free the used memory.

This approach of synchronization will be later referred to as direct task creation, as all the tasks are created directly.

■ **Listing 18** Task synchronization code of the Gemm Task

```
void GemmTask(void *params) {
    TaskHandle_t currentTaskHandle = xTaskGetCurrentTaskHandle();
    if(currentTaskHandle == layer1handle) {
        //Wait for all Taskhandles to be not NULL
        vTaskDelay(1);
    }
    else {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
    }
    ...
    if(currentTaskHandle == layer1handle) {
        xTaskNotifyGive(layer2handle);
    }
    else if(currentTaskHandle == layer4handle) {
        xTaskNotifyGive(layer5handle);
    }
    else {
        xTaskNotifyGive(layer8handle);
    }
    vTaskDelete(NULL);
}
```

The sequential execution of the layers through task notifications continues until the final layer concludes and signals the `predict_SimpleMLP` function to terminate its execution. An alternative method of synchronization is the use of semaphores or notifications between the creation of tasks. We will refer to this method later as separate task creation. This way, a task is created when the previous task is finished. Listing 19 illustrates this approach. Within the layer's task, the semaphore is released once computations are complete, allowing the other tasks to be created and resuming execution. This can also be implemented via notifications, where the `xSemaphoreTake` functions are replaced with `ulTaskNotifyTake` functions. Instead of giving the semaphore, the task functions send a notification to the main function, in this case `predict_SimpleMLP`.

■ **Listing 19** Semaphore synchronization example

```
void predict_SimpleMLP(double const *const x, double *pred) {
    xTaskCreate(GemmTask, "Layer1", 2048, &layer1params, 1, NULL);
    xSemaphoreTake(semaphore, portMAX_DELAY);
    xTaskCreate(BatchNormalizationTask, "Layer2", 2048,
        &layer2params, 1, NULL);
    xSemaphoreTake(semaphore, portMAX_DELAY);
    xTaskCreate(ReluTask, "Layer3", 2048, &layer3params, 1, NULL);
}
```

4.5 Batching Implementation

This section focuses on implementing batching, located in the benchmark function of the code structure, which enables testing the neural network with a larger volume of test data than was possible in the original setup. Batching will also allow us to test multi-threading

with batching, where two batches are executed concurrently. We will begin by examining the original implementation and the limitations it inherits and after that, we will present the batching implementation.

After the neural network has been trained from the FastInference tool, a `testing.csv` with 10.000 lines of testing data is provided. The benchmark function reads a specified number of lines from this CSV file and passes them to the neural network function for testing, evaluating its accuracy based on the output. The `read_csv` function, which is called by the benchmark function, is responsible for reading the `testing.csv` by using two vectors to load test data line by line, returning it as a tuple, as shown in listing 20. With each line read, these vectors dynamically expand in memory, resulting in a substantial vector object returned to the benchmark function, especially when reading a large number of lines. RAM is typically limited in embedded systems, and when a neural network model also occupies a significant portion of available memory, the memory for the test data object is increasingly small. This results in a significant limitation in the amount of test data we can utilize, before running out of memory while trying to execute our neural network model.

■ **Listing 20** The original `read_csv` function, highlighting the expanding of the vector object

```
auto read_csv(std::string &path) {
    std::vector<std::vector<double>> X;
    std::vector<unsigned int> Y;
    ...
    std::vector<double> x;
    while (std::getline(ss, entry, ',')) {
        if (i == label_pos) {
            Y.push_back(static_cast<unsigned int>(
                std::stoi(entry)));
        } else {
            x.push_back(static_cast<double>(std::stof(entry)));
        }
        X.push_back(x);
    }
    return std::make_tuple(X,Y);
}
```

The solution to this limitation is the introduction of batching, dividing the total test data into equal batches that are executed separately. This approach allocates RAM for only one batch at a time rather than the entire dataset, making the batch size the primary memory constraint. This enables testing of the entire dataset in theory, provided a reasonable batch size.

Listing 21 highlights the most important changes to the `read_csv` function, while omitting other changes for clarity. The function now returns `void` instead of `auto` and defines multiple parameters to configure the batching process. The vectors are cleared in the beginning of the function, to free the claimed memory and delete the data from the previous batch. The most noteworthy change is the for-loop, which limits the amount of lines to be read to the specified batching size.

■ **Listing 21** The batching version of the `read_csv` function

```
void read_csv(std::string &path, unsigned int batchSize,
    unsigned lineNumber, unsigned maxLine,
    std::vector<std::vector<double>> &X, std::vector<unsigned int> &Y ) {
    X.clear();
    Y.clear();
```

```

...
for(int batchSize = 1; batchSize <= batchingSize; batchSize++) {
    std::getline(file, line);
    while (std::getline(ss, entry, ',')) {
        if (i == label_pos) {
            Y.push_back(static_cast<unsigned int>(std::stoi(entry)));
        } else {
            x.push_back(static_cast<double>(std::stof(entry)));
        }
    }
    X.push_back(x);
}
}
}

```

4.6 Multithreading with FreeRTOS

This section examines multithreading in FreeRTOS to enhance the neural networks performance on our dual-core embedded system. We explore two strategies: multithreading by batching, where two batches are processed concurrently, and multithreading by layers, which distributes computation in demanding layers for parallel processing.

4.6.1 Multithreading by batching

Multithreading by batching is a technique used to improve processing efficiency by dividing a large dataset into smaller, manageable batches, each of which is processed in parallel. In neural network inference, this approach allows multiple batches of data to be handled simultaneously across threads, reducing the time required for extensive computations. By distributing two batches to a separate core, the workload is balanced, maximizing the available processing power and enabling higher throughput. To achieve this, we need to adjust the benchmark function and the neural network model itself. First, the benchmark function must be split into two tasks: one assigned to the first core and the other to the second core. These can then be called by the main function, which starts the benchmark process.(listing 22) The two benchmark tasks are functionally identical, differing only in their for-loops, which control which lines to read.(listing 23) The benchmark task on the first core reads the first half of the lines, while the task on the second core reads the second half.

■ **Listing 22** The main function creates two separate benchmark tasks

```

void app_main(void){
    ...
    xTaskCreatePinnedToCore(benchmark, "BenchmarkTask1",
        5000, &params1, 1, &benchmarkTaskHandle1, 0);
    xTaskCreatePinnedToCore(benchmark2core, "BenchmarkTask2",
        5000, &params1, 1, &benchmarkTaskHandle2, 1);
    ...
}

```

■ **Listing 23** The difference between the two benchmark tasks

```

void benchmark(void *params) {
    ...
    for(int testDataLine = 2;
        testDataLine <= lineNumbers/2; testDataLine += batchSize) {

```

```

        read_csv(path, batchSize, testDataLine, lineNumbers, X, Y);
        ...
    }
    void benchmark2core(void *params) {
        ...
        for(int testDataLine = (lineNumbers/2)+2;
            testDataLine <= lineNumbers; testDataLine += batchSize) {
            read_csv(path, batchSize, testDataLine, lineNumbers, X, Y);
            ...
        }
    }

```

Both benchmark tasks invoke the same neural network function, using the same model. To achieve this, the neural network implementation must be adjusted to avoid static TaskHandles and output arrays for the layers, as these could cause data corruption. For instance, if the two tasks both execute a neural network model with static output arrays, each task would interfere with the output, leading to potential data manipulation by both tasks. Therefore each static output array and static TaskHandle must be converted into local variables within the function itself, to ensure that each call of the neural network function produces independent copies for both tasks. This is demonstrated in listing 24, where the output arrays and TaskHandles, previously static and globally accessible, are now converted into local variables. The output arrays and the `taskHandleArray` are passed to the tasks through the layer specific task parameters.

■ **Listing 24** The static output arrays and Taskhandles turned into local variables in the neural network model

```

void predict_SimpleMLP15(double const *const x, double *pred) {
    double layer_1_output[8];
    double layer_2_output[8];
    ...

    TaskHandle_t layer1handle = NULL, layer2handle = NULL, ...;
    TaskHandle_t taskHandleArray[8] = {layer1handle, layer2handle, ...}
    GemmTaskParams layer1params{8,
                                784,
                                layer_1_output,
                                const_cast<double *>(layer_0_output),
                                layer_1_bias,
                                &layer_1_weight[0][0],
                                taskHandleArray};
}

```

4.6.2 Multithreading in layers

Multithreading by layers is an approach where the most computationally intensive layers of a neural network are distributed across both cores, enabling concurrent execution and reducing processing time. We focus solely on multithreading the computationally intensive layers, as certain layers execute so quickly that the added memory load and synchronization would introduce unnecessary overhead. For example, in our generated CNN model, the convolutional layer takes the longest to execute, while layers like MaxPool and BatchNormalization complete much more quickly.

To accomplish this, we can apply the same principle used in multithreading by batching, by splitting the benchmark task into two: one task processes the first half, while the other

handles the second half. Listing 25 shows the creation of two new convolution tasks, one assigned to core 0 and the other to core 1, each pinned to its respective core. The original `ConvTask` still is created, but it does no longer perform any computations. Instead, it manages synchronization of the two convolution tasks and signals the next task to continue execution. The `ConvTask` is reduced to identify which layer is running, waiting on the counting semaphore to be increased by the two convolutional tasks, and then signal the next layer to execute.(Listing 26)

■ **Listing 25** Creation of multithreaded tasks and pinning to cores

```
void predict_SmallCNN(double const * const x, double * pred) {
    xTaskCreate(ConvTask, "Layer2", 2000, NULL, 1, &layer2Handle);
    xTaskCreatePinnedToCore(ConvTaskCore0,
                            "ConvTaskCore0Layer2", 2000,
                            &layer2params, 1,
                            &layer2core0handle, 0);
    xTaskCreatePinnedToCore(ConvTaskCore1,
                            "ConvTaskCore1Layer2", 2000,
                            &layer2params, 1,
                            &layer2core1handle, 1);
}
```

■ **Listing 26** Convolution task now only with synchronization assignments

```
void ConvTask(void *params) {
    TaskHandle_t currentTaskHandle = xTaskGetCurrentTaskHandle();
    if(currentTaskHandle == layer6Handle) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        xTaskNotifyGive(layer6core0handle);
        xTaskNotifyGive(layer6core1handle);
    }
    else {
        //Waiting for all the other Taskhandle to load and be not NULL
        vTaskDelay(1);
        xTaskNotifyGive(layer2core0handle);
        xTaskNotifyGive(layer2core1handle);
    }
    xSemaphoreTake(convTaskSemaphore, portMAX_DELAY);
    xSemaphoreTake(convTaskSemaphore, portMAX_DELAY);
    if(currentTaskHandle == layer2Handle) {
        xTaskNotifyGive(layer3Handle);
    }
    else {
        xTaskNotifyGive(layer7Handle);
    }
    vTaskDelete(NULL);
}
```

5 Evaluation

This chapter focuses on evaluating the results of the implementations presented in the previous chapter. All raw benchmark results are available in the `Benchmark.txt` file within this [Github repository](#).

In the section 5.1 we explore how metrics such as runtime and peak heap memory usage are measured. We begin the benchmark section with 5.1.1, where we compare the task-based implementations to the original models. This examination focuses on the performance and memory impact that FreeRTOS tasks have on the neural network models, as well as comparing the different synchronization strategies, including two variants of task notifications and one semaphore strategy. Next, sub-section 5.1.2 evaluates the impact of batching on the testing process. After that, sub-section 5.1.3 compares the two multithreading approaches. To conclude the Benchmark section, sub-section 5.1.4 compares the runtime impact on the feed-forward model when another neural network is executing concurrently. In section 5.2, based on the results of the benchmark section, the advantages and drawbacks of using FreeRTOS tasks in the context of neural networks are discussed.

5.1 Performance & memory benchmarks

A commonly used metric in these benchmarks is the measured runtime of the models. We define runtime as the entire process of reading the batch data and testing the neural network with it. Once all data lines have been processed, the benchmark function sends a notification to the main function, which then begins calculating the metrics. Listing 27 illustrates this and the used metrics inside the main function. The `runtime` and `latencyPerElement` metrics are measured in milliseconds. The `min_free_heap_size` variable measures heap memory usage by indicating the smallest amount of available heap memory in bytes at any given point during the execution. A more memory-intensive neural network will result in a lower value.

Listing 27 Calculation of the metrics used for the benchmarks.

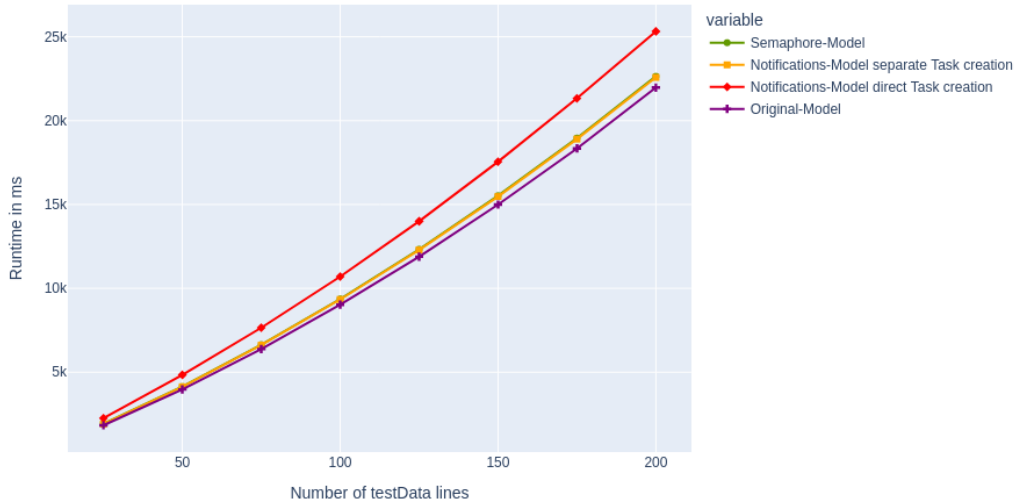
```
auto start = std::chrono::high_resolution_clock::now();
xTaskCreate(benchmark, "BenchTask", 8000, &params1, 1, &benchmark);
ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
auto end = std::chrono::high_resolution_clock::now();
auto accuracy = static_cast<float>(matches)/(lineNumbers - 1) * 100.f;
auto runtime = std::chrono::duration_cast<std::chrono::milliseconds>(
    end-start).count();
auto latencyPerElement = runtime / ((linenumbers-1)*repeat);
size_t min_free_heap_size = xPortGetMinimumEverFreeHeapSize();
```

5.1.1 Task-based implementations

In this section we examine the runtime and memory difference between the original and the task-based neural network models. In this benchmark, the `original model` is the simple feed-forward neural network we generated, with its implementation generated from the FastInference tool. The task-based models are implemented in three variations, each detailed in section 4.4 of the implementation chapter. `Semaphore-model` creates tasks separately, using a semaphore to synchronize them so that each task starts only after the previous one completes. The `Notifications-model separate Task creation` has the same principle, but utilizes notifications instead of semaphores. In contrast, the `Notifications-model`

`direct Task creation` utilizes a different approach, where all the tasks are created upfront and signal each other via notifications when the next task can begin execution. All models are tested with batching, using a batch size of 5 lines per batch. This approach is necessary, because otherwise, the maximum number of lines we could test would be limited to 25. Figure 9 shows the runtime difference between all models to up to 200 lines of test data. It is evident that the implementation of tasks introduces a slight increase in runtime due to the overhead of creating and managing the tasks. The runtime difference becomes more pronounced when testing larger datasets, such as 200 lines, as the increased number of batch executions favors the faster original model. The task-based model with the largest runtime increase is the notifications model with direct task creation. This may be due to the delay in initializing all task handles and the additional code required to identify the next layer. This results in an approximately 18.75% runtime increase in comparison to the original model. Both models that create tasks separately perform almost identically, with less than a 1% difference in runtime. Notably, the notifications model is slightly faster than the semaphore model, although the difference is less than 1%. Both of these models show an approximate 3% increase in runtime compared to the original model.

The memory difference between the notifications model with direct task creation and the two models with separate task creation is noteworthy, as I initially expected the models with separate task creation to be less memory-intensive. However, the opposite is true: the model with direct task creation uses 7% less heap memory at its peak usage.



■ **Figure 9** Comparison of runtime of the original- and task-based model

5.1.2 Batching

This section revolves around the results of the Batching implementation and demonstrating the severe limitations of the original test setup. Figure 10 compares three different execution strategies of the original feed-forward model we created. The model with the shortest runtime is clearly the original model, which processes all test lines in a single run. Following this, the model with the next lowest runtime is the batching model with a variable batch size,

which adapts the batch size to match the number of test lines. For example, when there are 20 test lines, the batch size is set to 20, requiring only one pass. This approach serves as a direct comparison to the original model, highlighting the additional runtime even when both strategies complete in a single run. The variable batching model also demonstrates that a higher batch size generally results in lower runtime, as seen when compared to the batching model with a fixed batch size of 5. Although a larger batch size leads to reduced runtime, it is constrained by the available heap memory, which is ultimately determined by the size of the neural network and its implementation.

Based on the benchmark data, we can conclude that the batching implementation results in approximately a 15% increase in runtime. This increase would likely be more significant with a larger volume of test data lines and fixed batch sizes. Although the increase in runtime is significant, the batching implementation is essential because the original testing implementation can only test a maximum of 28 lines, in case of the small feed-forward model. Beyond this limit, execution fails due to memory and vector-related issues. To more accurately capture the impact of the implementation features, all benchmarks are performed with the batching feature enabled, using a default batch size of 5.

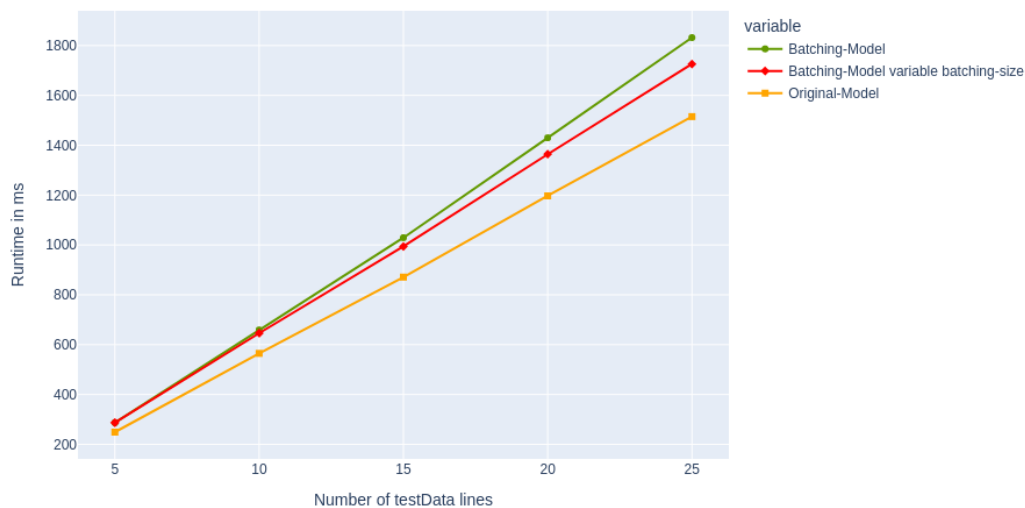


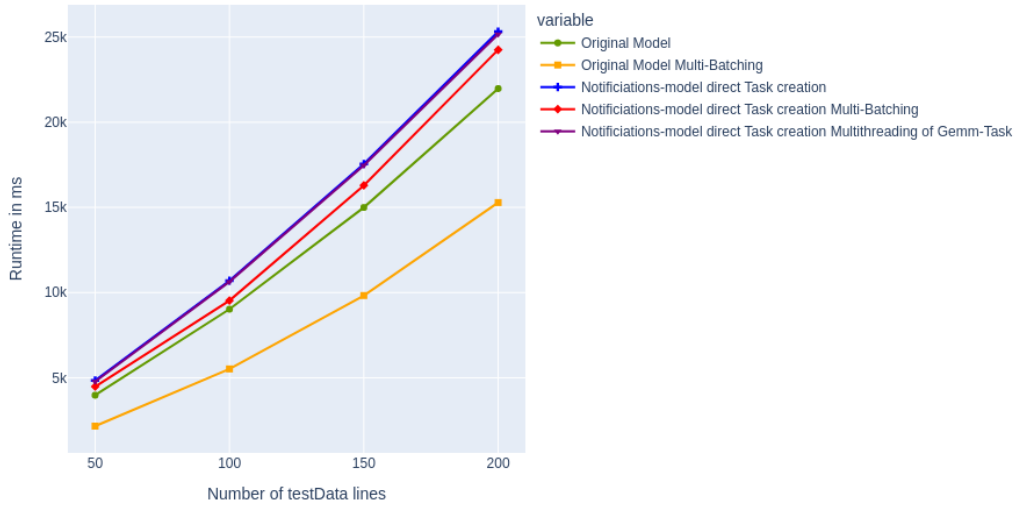
Figure 10 Comparison of runtime of the original feed-forward model, the feed-forward model with batching size of 5, and with a variable batching size

5.1.3 Multithreading – Batching and Layers

This section will cover the results of both multithreading approaches.

The first model we will discuss is the simple feed-forward network. For this model, it is possible to implement both multithreading within individual layers and concurrent execution of two batches. Figure 11 displays the results of these multithreading implementations, the baseline implementations they build upon on and the original model executed with multi-Batching. The only runtime improvement in comparison to the original implementation has the original model with multi-batching enabled. This results in a mean 37% runtime improvement over the original model. The model that most closely approached the performance of the

original model was the notifications model with direct task creation and multi-batching. However, the added runtime and overhead from the task-based implementation cannot be fully eliminated, resulting in a 9% slower performance compared to the original model. The multi-batching solution outperforms its counterpart that uses multithreading within layers. The multithreading version performs the Gemm-tasks on two cores concurrently, but it is only able to achieve a 1% runtime improvement over the base notifications version it is based upon. This is due to the computational intensity of the tasks. The Gemm task, while the most computation-heavy in this model, remains relatively balanced compared to the other tasks. In contrast, the convolution task in the CNN network is by far the most computationally demanding and will benefit significantly from concurrent execution on two cores, as we will explore later in this section.

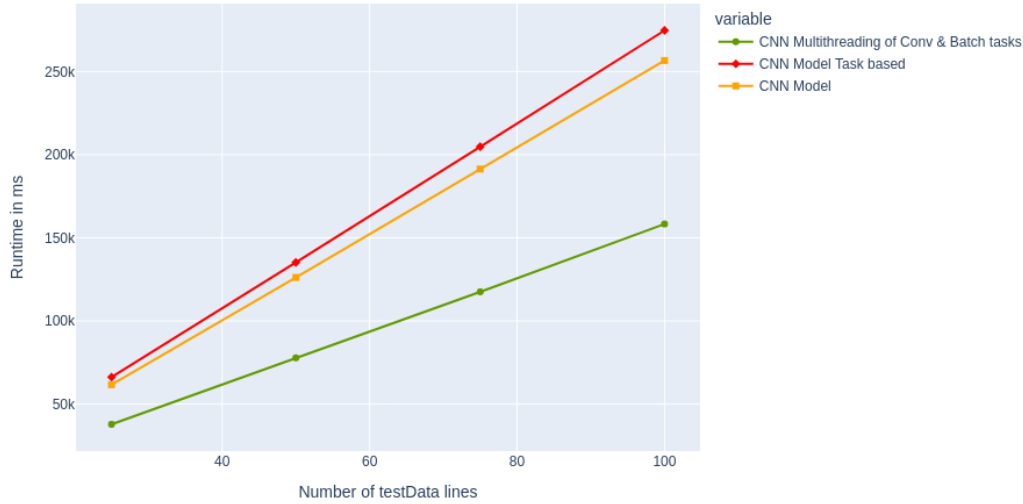


■ **Figure 11** Runtime comparison of multithreaded and multi-batching versions of the simple feed-forward network as the original model.

For the generated CNN, the multithreading batching approach cannot be executed due to memory constraints, as storing all output arrays in the heap requires excessive space and results in exceptions when running. However, multithreading by layers is feasible and provides a significant runtime improvement by leveraging parallel processing. Figure 12 illustrates the results of this. The `CNN-Model` is the original model we created from the FastInference tool. The `CNN-model Task based` is a implementation working with notifications and direct task creation, identical to the simple feed-forward version of the same approach. Lastly, `CNN multithreading of Conv & batch tasks` utilizes also a notification and direct task creation approach, with convolution and batch normalization tasks parallelized to run concurrently on two cores.

The task-based implementation of the CNN model results in a relatively small 7% runtime increase in comparison to the original CNN model. Although the runtime increase from implementing FreeRTOS tasks did not seem justified initially, the multithreaded version of the CNN model, which is built on the task-based model, demonstrates its benefits. By computing the convolution and batch normalization tasks on two cores simultaneously, this model achieves a 38% runtime improvement compared to the original model. This runtime

improvement comes with an increased memory usage, as the free heap memory at its peak usage decreases by 34% in comparison to the original model.



■ **Figure 12** Runtime comparison of the original CNN model, the task-based CNN model with notifications, and the multithreaded variant of the CNN model

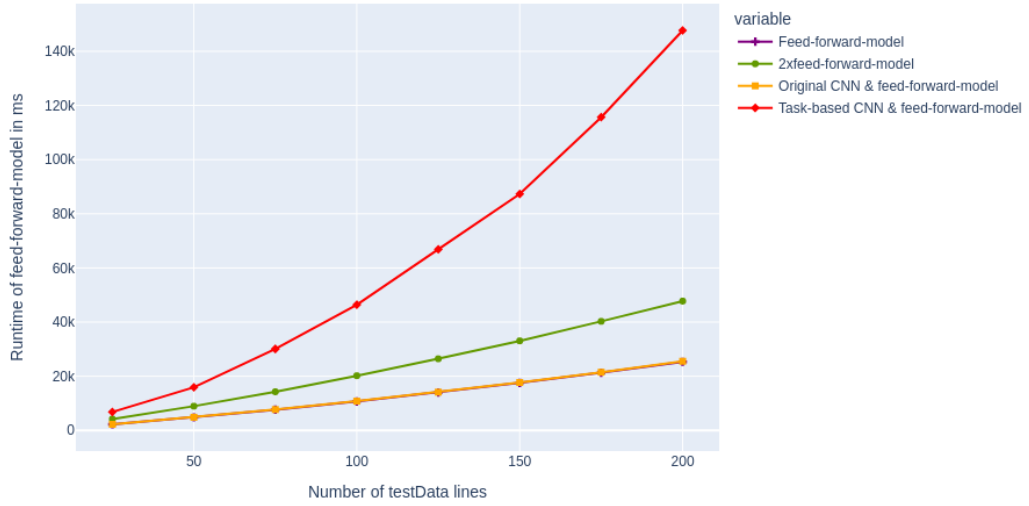
5.1.4 Concurrent Execution of two separate models

In this section we examine the runtime impact on our generated feed-forward network when a separate neural network is executed concurrently. The implementation is similar to multithreading by batching, except that two benchmark tasks now invoke two separate neural network functions, each for a different model. Both neural network models test the full number of test data lines. The feed-forward network is tested alongside three different neural networks executing simultaneously, with each task having the same priority. It is important to note that all the presented results are the runtime of the feed-forward model alone, not the total runtime of both models. Moreover, the feed-forward-model used in all scenarios uses direct task creation and synchronization via notifications. Figure 13 presents the results of the three different scenarios and shows the **feed-forward-model** as the baseline for comparison to the execution of the model without another concurrent neural network executing.

The first scenario we examine is the **2xfeed-forward-model**, which involves executing two feed-forward-models concurrently. In this case, the additional FreeRTOS tasks that the scheduler must manage result in a average 87% increase in runtime compared to the single model execution. Since the models and their task creation are identical, the scheduler can efficiently utilize both cores and distribute their concurrent execution.

The following two scenarios involve a smaller version of the binarized CNN we generated, as the original was too memory intensive to run concurrently with the feed-forward model. The CNN still consumes significantly more memory and runtime than the feed-forward-model. In the case of the **Original CNN & feed-forward-model**, the original CNN was executed concurrently without any task-based implementation. As no additional tasks

are added for the scheduler to manage beside the feed-forward model, the runtime of the feed-forward model is nearly identical to its single execution. This changes significantly when the CNN is also executed with a task-based approach, as seen with the **Task-based CNN & feed-forward-model** result. In this case, the runtime of the feed-forward model increases by an average of 341% compared to the execution of the original CNN & the feed-forward-model. This drastic increase in runtime is due to the scheduler also handling the CNN tasks, which take significantly longer to execute than the tasks of the feed-forward-model, thereby slowing the feed-forward model down. Once again, these results reflect only the runtime of the feed-forward model and not the total runtime of both models. For example, the total runtime of the **Task based CNN & feed-forward-model** compared to the **Original CNN & feed-forward model** increases by just an average of 10%, while the runtime of the feed-forward model increases significantly.



■ **Figure 13** Runtime Comparison of a feed-forward network across four scenarios. This benchmark exclusively reflects the runtime of the feed-forward network and does not include the runtime of both neural networks.

5.2 Evaluating the Pros and Cons of FreeRTOS Tasks Implementations

The previous section presented the results of the various implementations of FreeRTOS tasks across different scenarios. Based on these findings, we now examine the advantages and drawbacks of FreeRTOS tasks in the context of neural network inference.

We begin by examining the advantages and ideal scenarios for FreeRTOS tasks implementations. The most effective scenario was achieved by multithreading the CNN layers, leading to a significant runtime improvement. FreeRTOS facilitates this by allowing straightforward task creation and, through core pinning, enables direct control over which cores execute specific tasks. Concurrent task execution is essential for optimizing neural network performance with FreeRTOS and it is likely FreeRTOS's greatest advantage. Additionally, isolating each layer into individual tasks simplifies parameter adjustments, as only the task parameters need to be modified, leaving the layer code unchanged, which also reduces code redundancy.

Another benefit of FreeRTOS is its scheduler and the simplicity with which code or entire components can be added for execution through tasks. As demonstrated in section 5.1.4, where the addition of a second neural network model was simple and automatically utilized the second core of the ESP-32, which lead to an efficient throughput increase.

These advantages come with certain drawbacks that need to be considered when choosing to utilize FreeRTOS tasks. The first drawback is the increased memory usage required for the tasks and their control structures as highlighted by the results of the multithreading the CNN layers, where a 38% improvement in runtime was accompanied by a 34% increase in heap memory usage. Although this can be mitigated by limiting execution to one task at a time, especially in neural networks where only a single layer needs to run at any given moment, this memory demand remains an important consideration. Another drawback is the overhead introduced by synchronization and task creation, which can impact runtime performance. This is also reflected in the results shown in figure 9, where the different synchronization approaches were compared to the original-model, each showing an increase in runtime. Additionally, developers must carefully manage synchronization to avoid deadlocks, adding complexity to task implementation. The priority setting of individual tasks must also be considered, as the results of 5.1.4 showed, where the runtime of a component varied significantly when all tasks were given the same priority, depending on how many tasks from other components also needed to be managed by the scheduler.



6 Conclusion

This chapter provides a summary of the thesis and its findings, along with an outlook on potential future work in this field.

6.1 Summary

In this bachelor's thesis, FreeRTOS tasks have been implemented to represent layers of neural networks. Task execution was managed using three different synchronization strategies: two with task notifications and one with semaphores. A batching solution was implemented for the testing process, because the original testing strategy was constrained by a low number of testing lines before exhausting available memory. The impact of these implementations was tested on a simple feed-forward model and a binarized CNN, adapted to fit the ESP-32's memory constraints. Chapter 5 presented the runtime and memory results, demonstrating that FreeRTOS can enhance neural network runtime through multithreading. The extent of this improvement varied by neural network model, with the CNN showing significantly greater runtime gains than the feed-forward network due to the high computational demand of the convolutional layer. Consequently, the CNN benefited substantially from parallelizing its most intensive tasks across two cores. This 38% improvement in runtime was accompanied by a 34% increase in heap memory usage, demonstrating that implementing FreeRTOS tasks demands higher memory consumption, a critical consideration given the limited memory resources in embedded systems. Multithreading the most intensive layers of the feed-forward network did not yield a runtime improvement over the original implementation without tasks, indicating that the overhead introduced by the task implementation can sometimes outweigh the benefits of concurrent task execution.

Multithreading by executing two batches as tasks concurrently was tested with the feed-forward network model, resulting in a runtime improvement ranging from just 1% to 37% compared to the implementations the multithreaded versions are based upon. The concurrent execution of two separate neural network demonstrated how the runtime of one model can vary, based on the number of tasks created by the the other neural network model and their computational cost.

6.2 Outlook for future works

Building on the findings presented in this thesis, there are several promising topics for future research and enhancements. One potential topic would be to analyze the execution of a neural network with FreeRTOS tasks in a real-world application setting. This would offer valuable insights into the practical performance impact of the different implementations of FreeRTOS tasks. Additionally, integrating external application code, that includes for example external sensors or communication protocols, into these systems could reveal how the multithreaded neural network handles delays while striving to complete time-critical tasks on schedule. This is based on sub-section 5.1.4, where the runtime of the feed-forward model varied, depending on how many other tasks needed to be scheduled.

Another potential enhancement could be the use of cooperative scheduling offered by FreeRTOS, which is restricted within the ESP-IDF framework used in this thesis. Cooperative scheduling could be well-suited for the sequential execution of neural networks, as each layer can yield control upon completion, potentially optimizing task switches.

Optimizing memory management to reduce the observed increase in heap memory usage during task implementations could be another valuable enhancement. Given the limited

memory available in embedded systems, exploring more efficient memory allocation strategies may allow for the utilization of larger, more accurate neural networks. Moreover, FreeRTOS offers other synchronization techniques beyond task notifications and semaphores that were not explored in this thesis. The impact of utilizing message queues and event groups could enhance the efficiency of neural network inference on embedded systems.



A Appendix

A.1 Difficulties

During the implementation phase of this thesis, several challenges emerged, requiring careful consideration and time. Key difficulties encountered include:

Memory Scarcity

One of the most persistent challenges was managing memory scarcity on the ESP32. Especially the larger CNN demanded extensive optimization efforts and had to be resized multiple times due to the high memory demands of multithreading. Additionally, the initial task stack sizes were set conservatively to avoid stack overflows. Over time, and through repeated testing, these stack sizes were significantly reduced, freeing up heap memory that could be reallocated to enhance the accuracy of the CNN slightly. Detecting and pinpointing memory leaks presented a challenge as well. For example, I initially forgot to delete a semaphore after the neural network model execution, which during batching execution, led to the creation of a new semaphore with each run, ultimately causing a memory leak.

Race Conditions and Deadlocks

Managing synchronization between the tasks posed another difficulty. The introduction of multithreading resulted in race conditions and occasional deadlocks, some of which proved difficult to resolve due to their seemingly random occurrence. Determining whether the ESP32 had frozen during execution or encountered a deadlock was also challenging.

FastInference on Windows

When attempting to generate neural network models with FastInference I encountered an issue from FastInference due to my Windows operating system. As it turned out, the model generation currently works only on Linux-based machines, as system commands for saving models and files do not function correctly on Windows. This was resolved by using the Windows Subsystem for Linux (WSL) terminal to run the tool.

ESP-IDF restricting access to FreeRTOSConfig.h

While exploring the FreeRTOS kernel documentation, I found the cooperative scheduling mode intriguing, as it seemed well-suited for the sequential execution of neural network model layers. Unfortunately, I discovered that the framework I used for developing the ESP32 implementations, ESP-IDF, restricts access to the configuration file needed to change the scheduling mode. The reason why this mode cannot be modified via the menu configuration is sadly not explained in the ESP-IDF programming guide.

A.2 Declaration of AI usage

In this thesis, artificial intelligence in the form of ChatGPT was used for support in translations, paraphrasing, and grammar checks. The AI assistance helped to ensure clarity and accuracy in the language usage, while the ideas, research, and overall content reflect my own work and analysis.

A.3 List of Figures

List of Figures

1	A multi-layer feed-forward neural network structure[6]	8
2	Example of a simple CNN architecture containing only the key layers	10
3	Example of ReLU activation followed by max-pooling for a 2x2 activation map	11
4	Task States transitions[16]	17
5	Image of the ESP-32-S3 system	20
6	Architecture of the feed-forward neural network.	21
7	The most important layers of the CNN. Created via the NN-SVG tool from Alexander LeNail.	21
8	Abstract illustration of the four main components of the code and the interactions with each other	22
9	Comparison of runtime of the original- and task-based model	32
10	Comparison of runtime of the original feed-forward model, the feed-forward model with batching size of 5, and with a variable batching size	33
11	Runtime comparison of multithreaded and multi-batching versions of the simple feed-forward network as the original model.	34
12	Runtime comparison of the original CNN model, the task-based CNN model with notifications, and the multithreaded variant of the CNN model	35
13	Runtime Comparison of a feed-forward network across four scenarios. This benchmark exclusively reflects the runtime of the feed-forward network and does not include the runtime of both neural networks.	36

A.4 List of Listings

Listings

1	Example task function[16]	13
2	xTaskCreate function[16]	13
3	Binary mutex creation function[16]	14
4	Mutex creation function[16]	14
5	Counting semaphore creation function[16]	15
6	Semaphore take function[16]	15
7	Semaphore give function[16]	15
8	Semaphore delete function[16]	15
9	Task Notification give method[16]	16
10	Task Notification take function[16]	16
11	Task delete function[16]	16
12	Task creation pinned to one core[16]	19
13	Original code of the gemm layer	23
14	Struct for the parameters when calling a Gemm task	23
15	First Task implementation of gemm layer	24
16	Task creation example	24
17	Task creation with Taskhandle	25
18	Task synchronization code of the Gemm Task	26
19	Semaphore synchronization example	26
20	The original read_csv function, highlighting the expanding of the vector object	27
21	The batching version of the read_csv function	27

22	The main function creates two separate benchmark tasks	28
23	The difference between the two benchmark tasks	28
24	The static output arrays and Taskhandles turned into local variables in the neural network model	29
25	Creation of multithreaded tasks and pinning to cores	30
26	Convolution task now only with synchronization assignments	30
27	Calculation of the metrics used for the benchmarks.	31

References

-
- References**
- 1 P. V. P. R. D. Sharuiev, "Research of the characteristics of a convolutional neural network on the esp32-cam microcontroller," vol. 28, no. 2, pp. 277487–1, 2023.
 - 2 F. Luna-Perejón, M. Domínguez-Morales, D. Gutiérrez-Galán, and A. Civit-Balcells, "Low-power embedded system for gait classification using neural networks," *Journal of Low Power Electronics and Applications*, vol. 10, no. 2, p. 14, 2020.
 - 3 S. Yao, Y. Zhao, H. Shao, S. Liu, D. Liu, L. Su, and T. Abdelzaher, "Fastdeepiot: Towards understanding and optimizing neural network execution time on mobile and embedded devices," in *Proceedings of the 16th ACM Conference on Embedded Networked Sensor Systems*, pp. 278–291, 2018.
 - 4 M. Merone, A. Graziosi, V. Lapadula, L. Petrosino, O. d'Angelis, and L. Vollero, "A practical approach to the analysis and optimization of neural networks on embedded systems," *Sensors*, vol. 22, no. 20, p. 7807, 2022.
 - 5 J. Chen, Y. He, Y. Zhang, P. Han, and C. Du, "Energy-aware scheduling for dependent tasks in heterogeneous multiprocessor systems," *Journal of Systems Architecture*, vol. 129, p. 102598, 2022.
 - 6 M. H. Sazli, "A brief review of feed-forward neural networks," *Communications Faculty of Sciences University of Ankara Series A2-A3 Physical Sciences and Engineering*, vol. 50, no. 01, 2006.
 - 7 A. D. Rasamoelina, F. Adjailia, and P. Sinčák, "A review of activation function for artificial neural network," in *2020 IEEE 18th World Symposium on Applied Machine Intelligence and Informatics (SAMi)*, pp. 281–286, 2020.
 - 8 A. Krogh, "What are artificial neural networks?," *Nature biotechnology*, vol. 26, no. 2, pp. 195–197, 2008.
 - 9 K. O'Shea, "An introduction to convolutional neural networks," *arXiv preprint arXiv:1511.08458*, 2015.
 - 10 N. Bjorck, C. P. Gomes, B. Selman, and K. Q. Weinberger, "Understanding batch normalization," in *Advances in Neural Information Processing Systems* (S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, eds.), vol. 31, Curran Associates, Inc., 2018.
 - 11 N. Aloysius and M. Geetha, "A review on deep convolutional neural networks," in *2017 International Conference on Communication and Signal Processing (ICCSP)*, pp. 0588–0592, 2017.
 - 12 T. Simons and D.-J. Lee, "A review of binarized neural networks," *Electronics*, vol. 8, no. 6, p. 661, 2019.
 - 13 M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1," 2016.
 - 14 W. Cedeno and P. A. Laplante, "An overview of real-time operating systems," *JALA: Journal of the Association for Laboratory Automation*, vol. 12, no. 1, pp. 40–45, 2007.

- 15 P. Hambarde, R. Varma, and S. Jha, “The survey of real time operating system: Rtos,” in *2014 International Conference on Electronic Systems, Signal Processing and Computing Technologies*, pp. 34–39, IEEE, 2014.
- 16 R. Barry, “Mastering the freertos™ real time kernel,” 2016.
- 17 E. Systems, *ESP32 ESP-IDF Programming Guide v5.2.3*. Espressif Systems, 2024.

Eidesstattliche Versicherung

(Affidavit)

Butzke, Kevin

Name, Vorname
(surname, first name)

225051

Matrikelnummer
(student ID number)

☒ Bachelorarbeit
(Bachelor's thesis)

☐ Masterarbeit
(Master's thesis)

Titel
(Title)

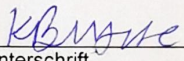
Hardware Aware Task Modeling of Neural Network Inference for FreeRTOS

Ich versichere hiermit an Eides statt, dass ich die vorliegende Abschlussarbeit mit dem oben genannten Titel selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

I declare in lieu of oath that I have completed the present thesis with the above-mentioned title independently and without any unauthorized assistance. I have not used any other sources or aids than the ones listed and have documented quotations and paraphrases as such. The thesis in its current or similar version has not been submitted to an auditing institution before.

Dortmund, 11.11.2024

Ort, Datum
(place, date)


Unterschrift
(signature)

Belehrung:

Wer vorsätzlich gegen eine die Täuschung über Prüfungsleistungen betreffende Regelung einer Hochschulprüfungsordnung verstößt, handelt ordnungswidrig. Die Ordnungswidrigkeit kann mit einer Geldbuße von bis zu 50.000,00 € geahndet werden. Zuständige Verwaltungsbehörde für die Verfolgung und Ahndung von Ordnungswidrigkeiten ist der Kanzler/die Kanzlerin der Technischen Universität Dortmund. Im Falle eines mehrfachen oder sonstigen schwerwiegenden Täuschungsversuches kann der Prüfling zudem exmatrikuliert werden. (§ 63 Abs. 5 Hochschulgesetz - HG -).

Die Abgabe einer falschen Versicherung an Eides statt wird mit Freiheitsstrafe bis zu 3 Jahren oder mit Geldstrafe bestraft.

Die Technische Universität Dortmund wird ggf. elektronische Vergleichswerkzeuge (wie z.B. die Software „turnitin“) zur Überprüfung von Ordnungswidrigkeiten in Prüfungsverfahren nutzen.

Die oben stehende Belehrung habe ich zur Kenntnis genommen:

Official notification:

Any person who intentionally breaches any regulation of university examination regulations relating to deception in examination performance is acting improperly. This offense can be punished with a fine of up to EUR 50,000.00. The competent administrative authority for the pursuit and prosecution of offenses of this type is the Chancellor of TU Dortmund University. In the case of multiple or other serious attempts at deception, the examinee can also be unenrolled. Section 63 (5) North Rhine-Westphalia Higher Education Act (*Hochschulgesetz, HG*).

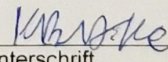
The submission of a false affidavit will be punished with a prison sentence of up to three years or a fine.

As may be necessary, TU Dortmund University will make use of electronic plagiarism-prevention tools (e.g. the "turnitin" service) in order to monitor violations during the examination procedures.

I have taken note of the above official notification.*

Dortmund, 11.11.2024

Ort, Datum
(place, date)


Unterschrift
(signature)

*Please be aware that solely the German version of the affidavit ("Eidesstattliche Versicherung") for the Bachelor's/ Master's thesis is the official and legally binding version.